ETAS VRTA V3.0.54

Status: Released

Port Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract. Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart.

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS VRTA V3.0.54 – Port Guide EN - 01-2025

Safety Notice

This ETAS product fulfills standard quality management requirements. If requirements of specific safety standards (e.g. IEC 61508, ISO 26262) need to be fulfilled, these requirements must be explicitly defined and ordered by the customer. Before use of the product, customer must verify the compliance with specific safety standards.

Contents

1 Introduction 9

1.1 About You 9

1.2 Document Conventions 10

1.3 References 10

2 Installing the RTA-OS Port Plug-in 11

2.1 Preparing to Install 11

2.1.1 Hardware Requirements 11

2.1.2 Software Requirements 11

2.2 Installation 11

2.2.1 Installation Directory 12

2.3 Licensing 12

2.3.1 Installing the ETAS License Manager 13

2.3.2 Licenses 14

2.3.3 Installing a Concurrent License Server 15

2.3.4 Using the ETAS License Manager 16

2.3.5 Troubleshooting Licenses 18

3 Verifying your Installation 20

3.1 Checking the Port 20

3.2 Running the Sample Applications 20

4 Port Characteristics 22

4.1 Parameters of Implementation 22

4.2 Run-time library 22

4.3 Configuration Parameters 23

4.3.1 Stack used for C-startup (SpPreStartOS) 23

4.3.2 Stack used when idle (SpStartOS) 23

4.3.3 Stack overheads for ISR activation (SpIDisp) 23

4.3.4 Stack overheads for ECC tasks (SpECC) 23

4.3.5 Stack overheads for ISR (SpPreemption) 24

4.3.6 ORTI22 (Orti22) 24
4.3.7 Number of Virtual Cores (CoreCount) 24
4.3.8 MultiCore interrupts (MC_Interrupt) 24
4.3.9 Run-time license name (LicName) 24
4.3.10 Instrument MinGW for Code Coverage (MinGWCoverage) 25
4.3.11 Enable stack repositioning (AlignUntrustedStacks) 25
4.4 Generated Files 25
5 Port-Specific API 26
5.1 Callbacks 26
5.1.1 Os_Cbk_ChangedTrust 26
5.1.2 Os_Cbk_ChangingTrust 27
5.1.3 Os_Cbk_GetSetProtection 28
5.1.4 Os_Cbk_SetMemoryAccess 29
5.2 Macros 33
5.2.1 CAT1_ISR 33
5.2.2 Os_DisableAllConfiguredInterrupts 33
5.2.3 Os_Disable_x 34
5.2.4 Os_EnableAllConfiguredInterrupts 34
5.2.5 Os_Enable_x 34
5.3 Type Definitions 34
5.3.1 Os_StackSizeType 34
5.3.2 Os_StackValueType 35
6 Toolchain 36
6.1 Compiler Versions 36
6.1.1 MinGW 36
6.1.2 VS2022 36
6.1.3 VS2019 37
6.1.4 VS2017 37
6.1.5 VS2015 37

6.1.6 VS2013 37
6.1.7 VS2012 37
6.2 Options used to generate this guide 37
6.2.1 Compiler (MinGW) 37
6.2.2 Librarian (MinGW) 38
6.2.3 Linker (MinGW) 38
6.2.4 Compiler (VS2010) 38
6.2.5 Librarian (VS2010) 39
6.2.6 Linker (VS2010) 39
6.2.7 Compiler (VS2012) 39
6.2.8 Librarian (VS2012) 39
6.2.9 Linker (VS2012) 39
6.2.10 Compiler (VS2015) 40
6.2.11 Librarian (VS2015) 40
6.2.12 Linker (VS2015) 40
6.2.13 Compiler (VS2017) 40
6.2.14 Librarian (VS2017) 41
6.2.15 Linker (VS2017) 41
6.2.16 Compiler (VS2019) 41
6.2.17 Librarian (VS2019) 42
6.2.18 Linker (VS2019) 42
6.2.19 Compiler (VS2022) 42
6.2.20 Librarian (VS2022) 43
6.2.21 Linker (VS2022) 43
7 Hardware 44
7.1 Supported Devices 44
7.2 Register Usage 44
7.2.1 Initialization 44
7.2.2 Modification 44

7.3 Required OS resources 45
7.4 Interrupts 45
7.4.1 Interrupt Priority Levels 45
7.4.2 Allocation of ISRs to Interrupt Vectors 45
7.4.3 Vector Table 45
7.4.4 Writing Category 1 Interrupt Handlers 46
7.4.5 Writing Category 2 Interrupt Handlers 46
7.4.6 Default Interrupt 46
7.5 Memory Model 47
7.6 Processor Modes 47
7.7 Stack Handling 47
8 Performance 48
8.1 Measurement Environment 48
8.2 RAM and ROM Usage for OS Objects 48
8.2.1 Single Core 48
8.2.2 Multi Core 49
8.3 Stack Usage 49
8.4 Library Module Sizes 50
8.4.1 Single Core 50
8.4.2 Multi Core 53
8.5 Execution Time 56
8.5.1 Context Switching Time 56
9 Finding Out More 59
10 Contacting ETAS 60
10.1 Technical Support 60
10.2 General Enquiries 60
10.2.1 ETAS Global Headquarters 60
10.2.2 ETAS Local Sales & Support Offices 60

1 Introduction

RTA-OS is a small and fast real-time operating system that conforms to both the AUTOSAR OS (3.0.1 -> 3.2.2, 4.0.1 -> 4.9.0 (R23-11)) and OSEK/VDX 2.2.3 standards (OSEK is now standardized in ISO 17356). The operating system is configured and built on a PC, but runs on your target hardware.

The RTA-OS tools are suitable for use in systems that conform to ASIL D.

This document describes the RTA-OS VRTA port plug-in that customizes the RTA-OS development tools for the Microsoft Windows Virtual ECU with the Visual Studio or MinGW C/C++ compiler. It supplements the more general information you can find in the **User Guide** and the **Reference Guide**.

The document has two parts. Chapters 2 to 3 help you understand the VRTA port and cover:

- how to install the VRTA port plug-in;
- how to configure VRTA-specific attributes;
- how to build an example application to check that the VRTA port plug-in works.

Chapters 4 to 8 provide reference information including:

- the number of OS objects supported;
- required and recommended toolchain parameters;
- how RTA-OS interacts with the Virtual ECU, including required register settings, memory models and interrupt handling;
- memory consumption for each OS object;
- memory consumption of each API call;
- execution times for each API call.

For the best experience with RTA-OS it is essential that you read and understand this document.

1.1 About You

You are a trained embedded systems developer who wants to build real-time applications using a preemptive operating system. You should have knowledge of the C programming language, including the compilation, assembling and linking of C code for embedded applications with your chosen toolchain. Elementary knowledge about your target microcontroller, such as the start address, memory layout, location of peripherals and so on, is essential.

You should also be familiar with common use of the Microsoft Windows operating system, including installing software, selecting menu items, clicking buttons, navigating files and folders.

1.2 Document Conventions

The following conventions are used in this guide:

Choose **File > Open**.	Menu options appear in **bold** characters.
Click **OK**.	Button labels appear in **bold** characters
Press <Enter>.	Key commands are enclosed in angle brackets.
The "Open file" dialog box appears	GUI element names, for example window titles, fields, etc. are enclosed in double quotes.
`Activate(Task1)`	Program code, header file names, C type names, C functions and API call names all appear in a `monospaced typeface`.
See Section 1.2.	Internal document hyperlinks are shown in blue letters.

1.3 References

OSEK is a European automotive industry standards effort to produce open systems interfaces for vehicle electronics. OSEK is now standardized in ISO 17356. For details of the OSEK standards, please refer to:

https://www.iso.org/standard/40079.html

AUTOSAR (AUTomotive Open System ARchitecture) is an open and standardized automotive software architecture, jointly developed by automobile manufacturers, suppliers and tool developers. For details of the AUTOSAR standards, please refer to:

http://www.autosar.org

2 Installing the RTA-OS Port Plug-in

2.1 Preparing to Install

RTA-OS port plug-ins are supplied as a downloadable electronic installation image which you obtain from the ETAS Web Portal. You will have been provided with access to the download when you bought the port. You may optionally have requested an installation DVD which will have been shipped to you. In either case, the electronic image and the installation DVD contain identical content.

> **NOTE**
>
> You must have installed the RTA-OS tools before installing the VRTA port plug-in. If you have not yet done this then please follow the instructions in the **Getting Started Guide**.

2.1.1 Hardware Requirements

You should make sure that you are using at least the following hardware before installing and using RTA-OS on a host PC:

- x86 PC
- 16 GB RAM
- 10 GB free disk space
- Network card (NIC)

2.1.2 Software Requirements

RTA-OS requires that your host PC has one of the following versions of Microsoft Windows installed:

- Windows® 10 64-bit
- Windows® 11 64-bit

RTA-OS supports the generation of the RTA-OS library under the following Linux® distribution:

- Ubuntu 22.04 64-bit

The library generation under Linux® requires additional support in the RTA-OS Target Port. Please check if this port supports generation under Linux®.

2.2 Installation

Target port plug-ins are installed in the same way as the tools:

1. Either
   - Double click the executable image; or

- Insert the RTA-OS VRTA CD into your CD-ROM or DVD drive.
  If the installation program does not run automatically then you will need to start the installation manually. Navigate to the root directory of your CD/DVD drive and double click `autostart.exe` to start the setup.

2. Follow the on-screen instructions to install the VRTA port plug-in.

By default, ports are installed into `C:\ETAS\RTA-OS\Targets`. During the installation process, you will be given the option to change the folder to which RTA-OS ports are installed. You will normally want to ensure that you install the port plug-in in the same location that you have installed the RTA-OS tools. You can install different versions of the tools/targets into different directories and they will not interfere with each other.

> **NOTE**
>
> Port plug-ins can be installed into any location, but using a non-default directory requires the use of the `--target_include` argument to both **RTAOSGen** and **RTAOSCfg**. For example:

```
RTAOSGen --target_include:<target_directory>
```

2.2.1 Installation Directory

The installation will create a sub-directory under `Targets` with the name `VRTA_3.0.54`. This contains everything to do with the port plug-in.

Each version of the port installs in its own directory - the trailing `_3.0.54` is the port's version identifier. You can have multiple different versions of the same port installed at the same time and select a specific version in a project's configuration.

The port directory contains:

VRTA.dll - the port plug-in that is used by **RTAOSGen** and **RTAOSCfg**.

RTA-OS VRTA Port Guide.pdf - the documentation for the port (the document you are reading now).

RTA-OS VRTA Release Note.pdf - the release note for the port. This document provides information about the port plug-in release, including a list of changes from previous releases and a list of known limitations.

There may be other port-specific documentation supplied which you can also find in the root directory of the port installation. All user documentation is distributed in PDF format which can be read using Adobe Acrobat Reader. Adobe Acrobat Reader is not supplied with RTA-OS but is freely available from http://www.adobe.com.

2.3 Licensing

RTA-OS is protected by FLEXnet licensing technology. You will need a valid license key in order to use RTA-OS.

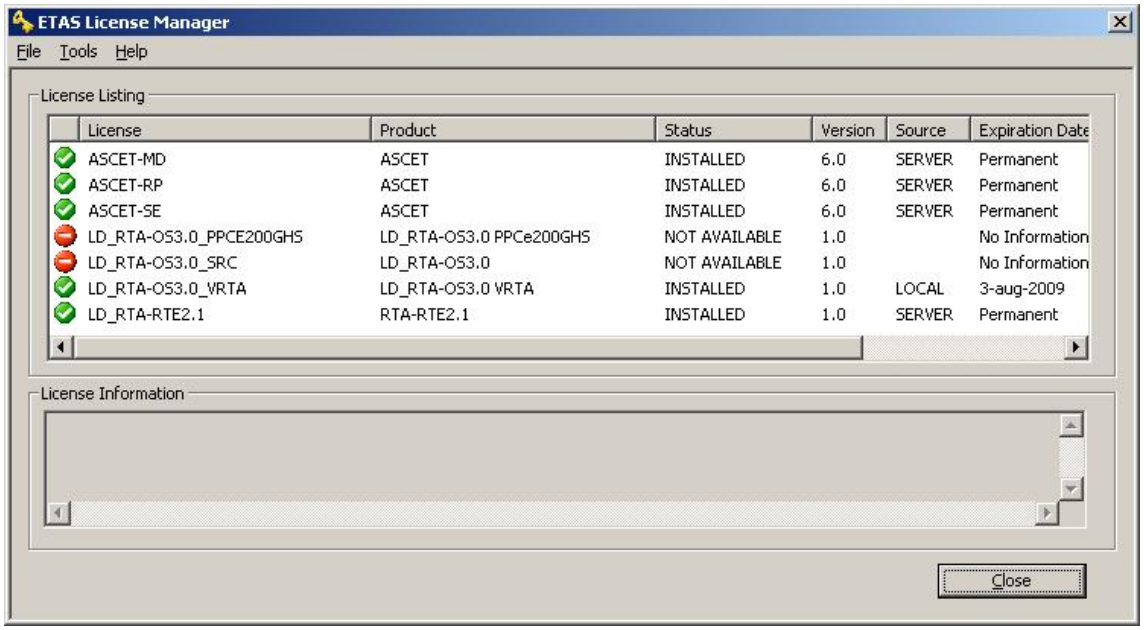


Figure 2.1: The ETAS License manager

Licenses for the product are managed using the ETAS License Manager which keeps track of which licenses are installed and where to find them. The information about which features are required for RTA-OS and any port plug-ins is stored as license signature files that are stored in the folder `<install_folder>\bin\Licenses`.

The ETAS License Manager can also tell you key information about your licenses including:

- Which ETAS products are installed
- Which license features are required to use each product
- Which licenses are installed
- When licenses expire
- Whether you are using a local or a server-based license

Figure 2.1 shows the ETAS License Manager in operation.

2.3.1 Installing the ETAS License Manager

> **NOTE**
>
> The ETAS License Manager must be installed for RTA-OS to work. It is highly recommended that you install the ETAS License Manager during your installation of RTA-OS.
> The latest version of the ETAS License Manager can be found by searching the ETAS download centre, or by visiting the following link: https://www.etas.com/lima and selecting 'License Manager' under Product/Topic.
> Please refer to the documentation inside the ETAS License Manager download package for the latest installation instructions.

The installer for the ETAS License Manager contains two components:

1. the ETAS License Manager itself;
2. a set of re-distributable FLEXnet utilities. The utilities include the software and instructions required to setup and run a FLEXnet license server manager if concurrent licenses are required (see Sections 2.3.2 and 2.3.3 for further details)

Once the installation is complete, the ETAS License Manager can be found in `C:\Program Files\Common Files\ETAS\Licensing`.

After it is installed, a link to the ETAS License Manager can be found in the Windows Start menu under **Programs➔ ETAS ➔ License Management ➔ ETAS License Manager**.

2.3.2 Licenses

When you install RTA-OS for the first time the ETAS License Manager will allow the software to be used in **grace mode** for 14 days. Once the grace mode period has expired, a license key must be installed. If a license key is not available, please contact your local ETAS sales representative. Contact details can be found in Chapter 10.

You should identify which type of license you need and then provide ETAS with the appropriate information as follows:

Machine-named licenses allows RTA-OS to be used by any user logged onto the PC on which RTA-OS and the machine-named license is installed.

A machine-named license can be issued by ETAS when you provide the host ID (Ethernet MAC address) of the host PC

User-named licenses allow the named user (or users) to use RTA-OS on any PC in the network domain.

A user-named license can be issued by ETAS when you provide the Windows user-name for your network domain.

Concurrent licenses allow any user on any PC up to a specified number of users to use RTA-OS. Concurrent licenses are sometimes called floating licenses because the license can float between users.

A concurrent license can be issued by ETAS when you provide the following information:

1. The name of the server
2. The Host ID (MAC address) of the server.
3. The TCP/IP port over which your FLEXnet license server will serve licenses. A default installation of the FLEXnet license server uses port 27000.

You can use the ETAS License Manager to get the details that you must provide to ETAS when requesting a machine-named or user-named license and (optionally) store this information in a text file.

Open the ETAS License Manager and choose **Tools ➔ Obtain License Info** from the menu. For machine-named licenses you can then select the network adaptor which provides the Host ID (MAC address) that you want to use as shown in Figure 2.2.

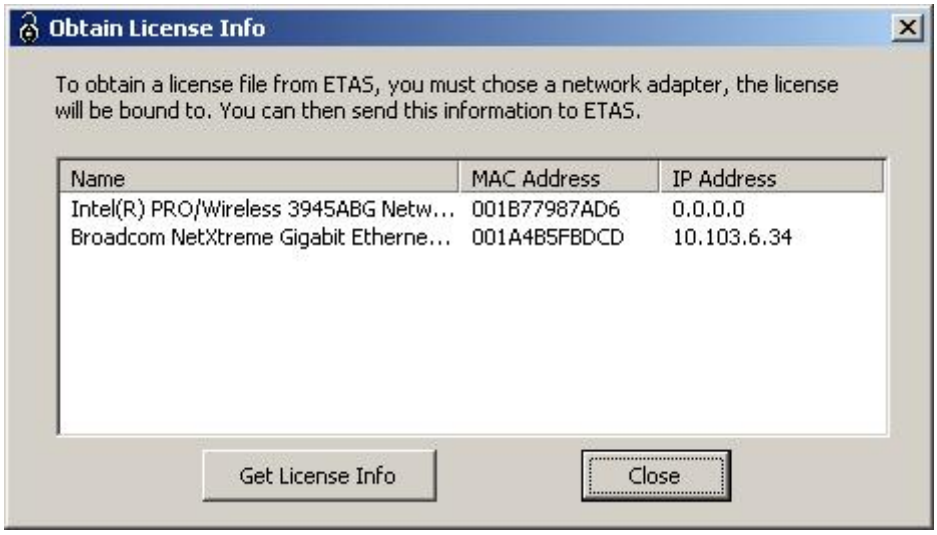


Figure 2.2: Obtaining License Information

For a user-based license, the ETAS License Manager automatically identifies the Windows username for the current user.

Selecting "Get License Info" tells you the Host ID and User information and lets you save this as a text file to a location of your choice.

2.3.3 Installing a Concurrent License Server

Concurrent licenses are allocated to client PCs by a FLEXnet license server manager working together with a vendor daemon. The vendor daemon for ETAS is called `ETAS.exe`. A copy of the vendor daemon is placed on disk when you install the ETAS License Manager and can be found in:

```
C:\Program Files\Common Files\ETAS\Licensing\Utility
```

To work with an ETAS concurrent license, a license server must be configured which is accessible from the PCs wishing to use a license. The server must be configured with the following software:

- FLEXnet license server manager;
- ETAS vendor daemon (`ETAS.exe`);

It is also necessary to install your concurrent license on the license server.

In most organizations there will be a single FLEXnet license server manager that is administered by your IT department. You will need to ask your IT department to install the ETAS vendor daemon and the associated concurrent license.

If you do not already have a FLEXnet license server then you will need to arrange for one to be installed. A copy of the FLEXnet license server, the ETAS vendor daemon and the instructions for installing and using the server (`LicensingEndUserGuide.pdf`) are placed on disk when you install the ETAS License manager and can be found in:

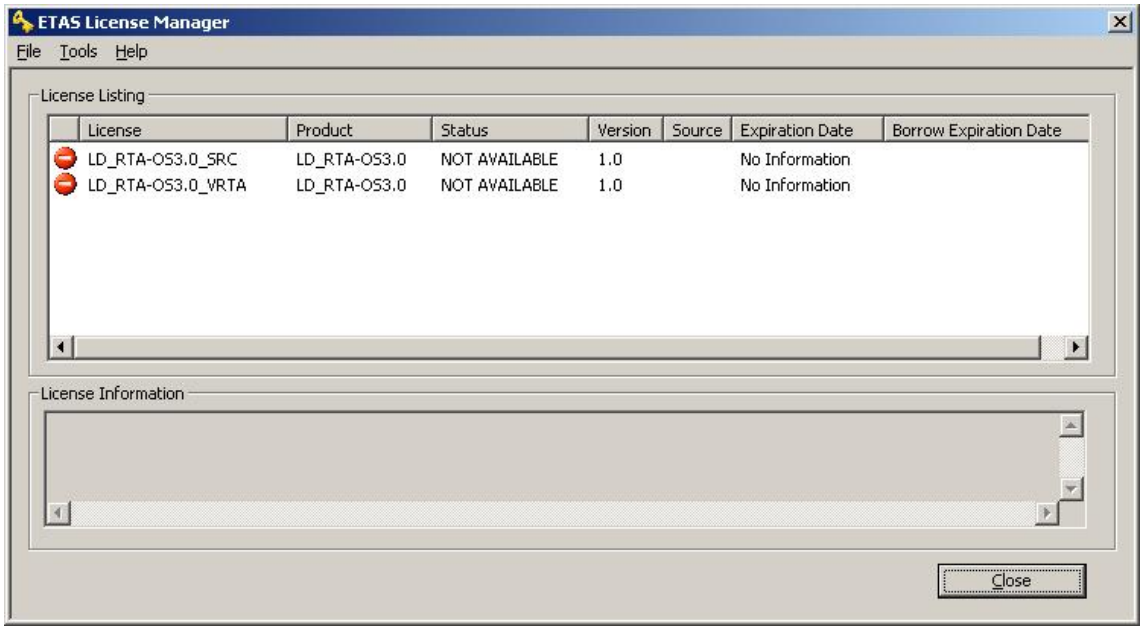


Figure 2.3: Unlicensed RTA-OS Installation

```
C:\Program Files\Common Files\ETAS\Licensing\Utility
```

2.3.4 Using the ETAS License Manager

If you try to run the RTA-OS GUI **RTAOSCfg** without a valid license, you will be given the opportunity to start the ETAS License Manager and select a license. (The command-line tool **RTAOSGen** will just report the license is not valid.)

When the ETAS License Manager is launched, it will display the RTA-OS license state as `NOT AVAILABLE`. This is shown in Figure 2.3.

Note that if the ETAS License Manager window is slow to start, **RTAOSCfg** may ask a second time whether you want to launch it. You should ignore the request until the ETAS License Manager has opened and you have completed the configuration of the licenses. You should then say yes again, but you can then close the ETAS License Manager and continue working.

2.3.4.1 License Key Installation

License keys are supplied in an ASCII text file, which will be sent to you on completion of a valid license agreement.

If you have a machine-based or user-based license key then you can simply install the license by opening the ETAS License Manager and selecting **File → Add License File** menu.

If you have a concurrent license key then you will need to create a license stub file that tells the client PC to look for a license on the FLEXnet server as follows:

1. create a copy of the concurrent license file

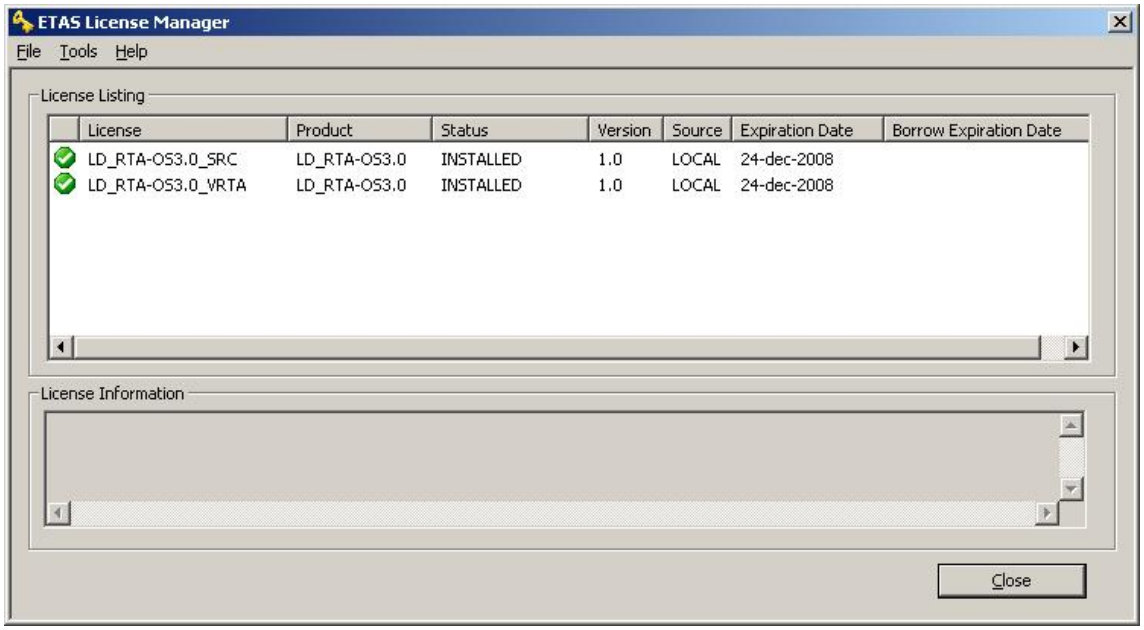


Figure 2.4: Licensed features for RTA-OS

2. open the copy of the concurrent license file and delete every line **except** the one starting with `SERVER`
3. add a new line containing `USE_SERVER`
4. add a blank line
5. save the file

The file you create should look something like this:

```
SERVER <server name> <MAC address> <TCP/IP Port>¶
USE_SERVER¶
¶
```

Once you have created the license stub file you can install the license by opening the ETAS License Manager and selecting **File → Add License File** menu and choosing the license stub file.

2.3.4.2 License Key Status

When a valid license has been installed, the ETAS License Manager will display the license version, status, expiration date and source as shown in Figure 2.4.

2.3.4.3 Borrowing a concurrent license

If you use a concurrent license and need to use RTA-OS on a PC that will be disconnected from the network (for example, you take a demonstration to a customer site), then the concurrent license will not be valid once you are disconnected.

To address this problem, the ETAS License Manager allows you to temporarily borrow a license from the license server.

To borrow a license:

1. Right click on the license feature you need to borrow.
2. Select "Borrow License"
3. From the calendar, choose the date that the borrowed license should expire.
4. Click "OK"

The license will automatically expire when the borrow date elapses. A borrowed license can also be returned before this date. To return a license:

1. Reconnect to the network;
2. Right-click on the license feature you have borrowed;
3. Select "Return License".

2.3.5 Troubleshooting Licenses

RTA-OS tools will report an error if you try to use a feature for which a correct license key cannot be found. If you think that you should have a license for a feature but the RTA-OS tools appear not to work, then the following troubleshooting steps should be followed before contacting ETAS:

Can the ETAS License Manager see the license?

The ETAS License Manager must be able to see a valid license key for each product or product feature you are trying to use.

You can check what the ETAS License Manager can see by starting it from the **Help ➔ License Manager...** menu option in **RTAOSCfg** or directly from the Windows Start Menu - **Start ➔ ETAS ➔ License Management ➔ ETAS License Manager**.

The ETAS License Manager lists all license features and their status. Valid licenses have status `INSTALLED`. Invalid licenses have status `NOT AVAILABLE`.

Is the license valid?

You may have been provided with a time-limited license (for example, for evaluation purposes) and the license may have expired. You can check that the Expiration Date for your licensed features to check that it has not elapsed using the ETAS License Manager.

If a license is due to expire within the next 30 days, the ETAS License Manager will use a warning triangle to indicate that you need to get a new license. Figure 2.5 shows that the license features `LD_RTA-OS3.0_VRTA` and `LD_RTA-OS3.0_SRC` are due to expire.

If your license has elapsed then please contact your local ETAS sales representative to discuss your options.

Does the Ethernet MAC address match the one specified?

If you have a machine based license then it is locked to a specific MAC address. You can find out the MAC address of your PC by using the ETAS License Manager (**Tools ➔ Obtain License Info**) or using the Microsoft program **ipconfig /all** at a Windows Command Prompt.

You can check that the MAC address in your license file by opening your license file in a text editor and checking that the HOSTID matches the MAC address

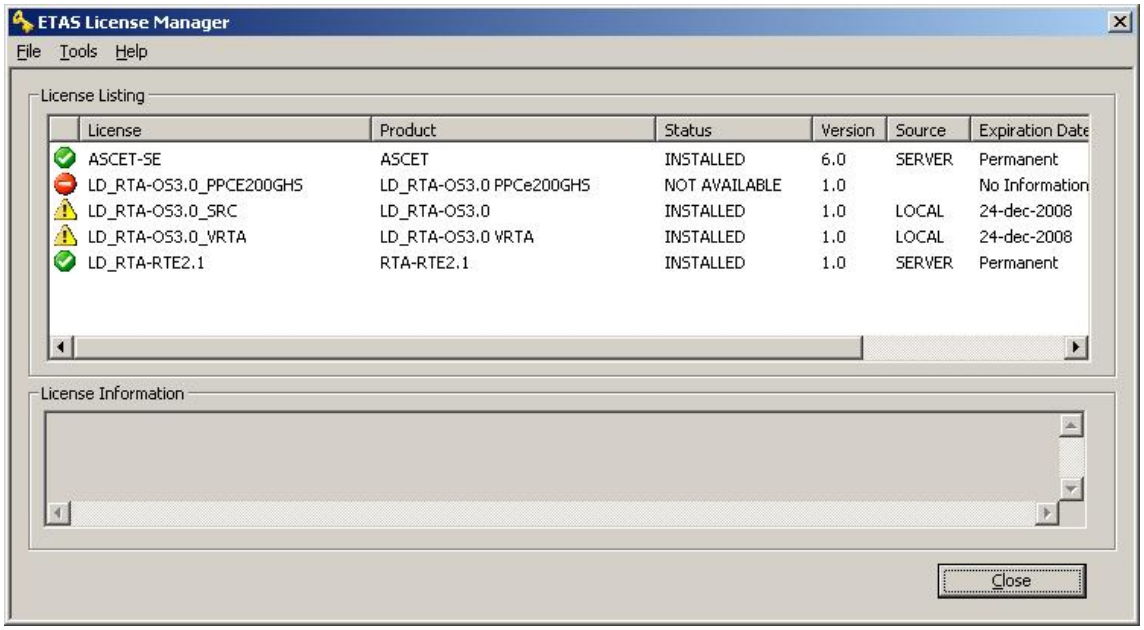


Figure 2.5: Licensed features that are due to expire

identified by the ETAS License Manager or the **Physical Address** reported by **ipconfig /all**.

If the HOSTID in the license file (or files) does not match your MAC address then you do not have a valid license for your PC. You should contact your local ETAS sales representative to discuss your options.

Is your Ethernet Controller enabled?

If you use a laptop and RTA-OS stops working when you disconnect from the network then you should check your hardware settings to ensure that your Ethernet controller is not turned off to save power when a network connection is not present. You can do this using Windows Control Panel. Select **System → Hardware → Device Manager** then select your Network Adapter. Right click to open **Properties** and check that the Ethernet controller is not configured for power saving in **Advanced** and/or **Power Management** settings.

Is the FlexNet License Server visible?

If your license is served by a FlexNet license server, then the ETAS License Manager will report the license as `NOT AVAILABLE` if the license server cannot be accessed.

You should contact your IT department to check that the server is working correctly.

Still not fixed?

If you have not resolved your issues, after confirming these points above, please contact ETAS technical support. The contact address is provided in Section 10.1. You must provide the contents and location of your license file and your Ethernet MAC address.

3 Verifying your Installation

Now that you have installed both the RTA-OS tools and a port plug-in and have obtained and installed a valid license key you can check that things are working.

3.1 Checking the Port

The first thing to check is that the RTA-OS tools can see the new port. You can do this in two ways:

1. use the **RTAOSGen** tool
   You can run the command **RTAOSGen** −−**target:?** to get a list of available targets, the versions of each target and the variants supported, for example:

```
RTA-OS Code Generator
Version p.q.r.s, Copyright © ETAS nnnn
Available targets:
 TriCoreHighTec_n.n.n [...,TC39xB]
 VRTA_n.n.n [MinGW,...,VS2019,VS2022]
```

2. use the **RTAOSCfg** tool
   The second way to check that the port plug-in can be seen is by starting **RTAOSCfg** and selecting **Help → Information...** drop down menu. This will show information about your complete RTA-OS installation and license checks that have been performed.

> **NOTE**
> If the target port plug-ins have been installed to a non-default location, then the `--target_include` argument must be used to specify the target location.

If the tools can see the port then you can move on to the next stage – checking that you can build an RTA-OS library and use this in a real program that will run on your target hardware.

3.2 Running the Sample Applications

Each RTA-OS port is supplied with a set of sample applications that allow you to check that things are running correctly. To generate the sample applications:

1. Create a new **working** directory in which to build the sample applications.
2. Open a Windows command prompt in the new directory.
3. Execute the command:

```
RTAOSGen --target:<your target> --samples:[Applications]

e.g.

RTAOSGen --target:[MPC5777Mv2]PPCe200HighTec_5.0.8
    --samples:[Applications]
```

You can then use the build.bat and run.bat files that get created for each sample application to build and run the sample. For example:

```
cd Samples\Applications\HelloWorld
build.bat
run.bat
```

Remember that your target toolchain must be accessible on the Windows PATH for the build to be able to run successfully.

> **NOTE**
>
> It is strongly recommended that you build and run at least the Hello World example in order to verify that RTA-OS can use your compiler toolchain to generate an OS kernel and that a simple application can run with that kernel.

For further advice on building and running the sample applications, please consult your **Getting Started Guide**.

4 Port Characteristics

This chapter tells you about the characteristics of RTA-OS for the VRTA port.

The VRTA port supports multicore operation on the devices listed later in the section on supported devices.

4.1 Parameters of Implementation

To be a valid OSEK (ISO 17356) or AUTOSAR OS, an implementation must support a minimum number of OS objects. The following table specifies the **minimum** numbers of each object required by the standards and the **maximum** number of each object supported by RTA-OS for the VRTA port.

Parameter	Required	RTA-OS
Tasks	16	1024
Tasks not in SUSPENDED state	16	1024
Priorities	16	1024
Tasks per priority	-	1024
Queued activations per priority	-	4294967296
Events per task	8	63
Software Counters	8	4294967296
Hardware Counters	-	4294967296
Alarms	1	4294967296
Standard Resources	8	4294967296
Linked Resources	-	4294967296
Nested calls to `GetResource()`	-	4294967296
Internal Resources	2	no limit
Application Modes	1	4294967296
Schedule Tables	2	4294967296
Expiry Points per Schedule Table	-	4294967296
OS Applications	-	4294967295
Trusted functions	-	4294967295
Spinlocks (multicore)	-	4294967295
Register sets	-	4294967296

4.2 Run-time library

The run-time library of the compiler toolchain is expected to provide the following functions for the VRTA port.

Function	Description
`memcpy`	Used for ECC Task context saving
`memset`	Used to initialize data during StartOS.

4.3 Configuration Parameters

Port-specific parameters are configured in the **General → Target** workspace of **RTAOSCfg**, under the "Target-Specific" tab.

The following sections describe the port-specific configuration parameters for the VRTA port, the name of the parameter as it will appear in the XML configuration and the range of permitted values (where appropriate).

4.3.1 Stack used for C-startup (SpPreStartOS)

Description

The amount of stack already in use at the point that StartOS() is called. This value is simply added to the total stack size that the OS needs to support all tasks and interrupts at run-time. Typically you use this to obtain the amount of stack that the linker must allocate. The value does not normally change if the OS configuration changes.

4.3.2 Stack used when idle (SpStartOS)

Description

The amount of stack used when the OS is in the idle state (typically inside Os_Cbk_Idle()). This is just the difference between the stack used at the point that Os_StartOS() is called and the stack used when no task or interrupt is running. This can be zero if Os_Cbk_Idle() is not used. It must include the stack used by any function called while in the idle state. The value does not normally change if the OS configuration changes.

4.3.3 Stack overheads for ISR activation (SpIDisp)

Description

The extra amount of stack needed to activate a task from within an ISR. If a task is activated within a Category 2 ISR, and that task has a higher priority than any currently running task, then for some targets the OS may need to use marginally more stack than if it activates a task that is of lower priority. This value accounts for that. On most targets this value is zero. This value is used in worst-case stack size calculations. The value may change if significant changes are made to the OS configuration. e.g. STANDARD/EXTENDED, SC1/2/3/4.

4.3.4 Stack overheads for ECC tasks (SpECC)

Description

The extra amount of stack needed to start an ECC task. ECC tasks need to save slightly more state on the stack when they are started than BCC tasks. This value contains the difference. The value may change if significant changes are made to

the OS configuration. e.g. STANDARD/EXTENDED, SC1/2/3/4. Also note that if you are using stack repositioning (to align the stack of untrusted code to suit the MPU) then you will need to reduce the value by the amount of the adjustment.

4.3.5 Stack overheads for ISR (SpPreemption)

Description

The amount of stack used to service a Category 2 ISR. When a Category 2 ISR interrupts a task, it usually places some data on the stack. If the ISR measures the stack to determine if the preempted task has exceeded its stack budget, then it will overestimate the stack usage unless this value is subtracted from the measured size. The value is also used when calculating the worst-case stack usage of the system. Be careful to set this value accurately. If its value is too high then when the subtraction occurs, 32-bit underflow can occur and cause the OS to think that a budget overrun has been detected. The value may change if significant changes are made to the OS configuration. e.g. STANDARD/EXTENDED, SC1/2/3/4.

4.3.6 ORTI22 (Orti22)

Description

Select an ORTI generation based on the the Lauterbach debugger.

Settings

Value	Description
true	Generate ORTI
false	No ORTI

4.3.7 Number of Virtual Cores (CoreCount)

Description

Specify the number of cores to create. Defaults to 1.

4.3.8 MultiCore interrupts (MC_Interrupt)

Description

Select interrupt reserved for multi-core implementation.

4.3.9 Run-time license name (LicName)

Description

Override the name of the run-time license. Default LD_RTA-OS_VRTA.

4.3.10 Instrument MinGW for Code Coverage (MinGWCoverage)

Description

Build with gcov code coverage on MinGW

Settings

Value	Description
true	Instrument for Code coverage
false	Normal compilation (default)

4.3.11 Enable stack repositioning (AlignUntrustedStacks)

Description

Use to support realignment of the stack for untrusted code when there are MPU protection region granularity issues. Only supported on VS2022 variant. Refer to the documentation for Os_Cbk_SetMemoryAccess.

Settings

Value	Description
true	Support repositioning
false	Normal behavior (default)

4.4 Generated Files

The following table lists the files that are generated by **RTAOSGen** for all ports:

Filename	Contents
`Os.h`	The main include file for the OS.
`Os_Cfg.h`	Declarations of the objects you have configured. This is included by `Os.h`.
`Os_MemMap.h`	AUTOSAR memory mapping configuration used by RTA-OS to merge with the system-wide `MemMap.h` file in AUTOSAR versions 4.0 and earlier. From AUTOSAR version 4.1, `Os_MemMap.h` is used by the OS instead of `MemMap.h`. This is an optionally generated sample.
`RTAOS.<lib>`	The RTA-OS library for your application. The extension `<lib>` depends on your target.
`RTAOS.<lib>.sig`	A signature file for the library for your application. This is used by **RTAOSGen** to work out which parts of the kernel library need to be rebuilt if the configuration has changed. The extension `<lib>` depends on your target.
`<projectname>.log`	A log file that contains a copy of the text that the tool and compiler sent to the screen during the build process.

5 Port-Specific API

The following sections list the port-specific aspects of the RTA-OS programmers reference for the VRTA port that are provided either as:

- additions to the material that is documented in the **Reference Guide**; or
- overrides for the material that is documented in the **Reference Guide**. When a definition is provided by both the **Reference Guide** and this document, the definition provided in this document takes precedence.

5.1 Callbacks

5.1.1 Os_Cbk_ChangedTrust

Called just after changing trust state.

Syntax

```
FUNC(void, {memclass})Os_Cbk_ChangedTrust(
    Os_tmaskType current_tmask
)
```

Description

By default, VRTA applications do not implement memory protection. They just keep track of the trust state that would be observed in a real target.

If you wish to add memory protection yourself then you can use this callback to determine when to switch protection maps.

Os_Cbk_ChangedTrust gets called just after the internal state is changed. It gets passed the trust state that is about to be set.

Note that in multi-core applications trust state is independent on each core.

If you wish to change data between Os_Cbk_ChangingTrust and Os_Cbk_ChangedTrust, you might consider using

vrtaEnterUninterruptibleSection / vrtaLeaveUninterruptibleSection to prevent preemption at the wrong time.

Note: memclass is OS_APPL_CODE for AUTOSAR 3.x, OS_OS_CBK_CHANGEDTRUST_CODE for AUTOSAR 4.1 and OS_CALLOUT_CODE otherwise.

Example

```
FUNC(void, {memclass}) Os_Cbk_ChangedTrust(Os_tmaskType
    current_tmask) {
  if (current_tmask == OS_USER_MODE) {
    /* Set mapping for user / untrusted */
  } else if (current_tmask == OS_TWP_MODE) {
    /* Set mapping for trusted with protection */
  }
}
```

Required when

Optional.

See Also

Os_Cbk_ChangingTrust

5.1.2 Os_Cbk_ChangingTrust

Called just before changing trust state.

Syntax

```
FUNC(void, {memclass})Os_Cbk_ChangingTrust(
    Os_tmaskType new_tmask
)
```

Description

By default, VRTA applications do not implement memory protection. They just keep track of the trust state that would be observed in a real target.

If you wish to add memory protection yourself then you can use this callback to determine when to switch protection maps.

Os_Cbk_ChangingTrust gets called just before the internal state is changed. It gets passed the trust state that is about to be set.

Note that in multi-core applications trust state is independent on each core.

If you wish to change data between Os_Cbk_ChangingTrust and Os_Cbk_ChangedTrust, you might consider using

vrtaEnterUninterruptibleSection / vrtaLeaveUninterruptibleSection to prevent preemption at the wrong time.

Note: memclass is OS_APPL_CODE for AUTOSAR 3.x, OS_OS_CBK_CHANGINGTRUST_CODE for AUTOSAR 4.1 and OS_CALLOUT_CODE otherwise.

Example

```
FUNC(void, {memclass}) Os_Cbk_ChangingTrust(Os_tmaskType new_tmask)
    {
  if (new_tmask == OS_SUPERVISOR_MODE) {
    /* Set mapping for supervisor / fully trusted */
  }
}
```

Required when

Optional.

See Also

Os_Cbk_ChangedTrust

5.1.3 Os_Cbk_GetSetProtection

Callback routine used to control the activation of the memory protection system.

Syntax

```
FUNC(boolean, {memclass})Os_Cbk_GetSetProtection(
    boolean enable
)
```

Return Values

The call returns values of type `boolean`.

Description

This callback is used in configurations that have OS Applications where TrustedApplicationWithProtection is true. It must return the state of the memory protection hardware at the point it was called (TRUE if enabled, FALSE otherwise). It must then enable or disable memory protection based on the incoming 'enable' value. It is used to switch between Trusted and TrustedApplicationWithProtection modes. It is only called when the system is in a Trusted or TrustedApplicationWithProtection mode.

The callback is required for this target platform. (Some platforms such as the TriCore can provide separate memory protection sets for untrusted, trusted and trusted-with-protection modes and in that case the callback is not used.)

Note: memclass is OS_APPL_CODE for AUTOSAR 3.x, OS_OS_CBK_GETSETPROTECTION_CODE for AUTOSAR 4.1 and OS_CALLOUT_CODE otherwise.

Example

```
FUNC(boolean {memclass}) Os_Cbk_GetSetProtection(boolean enable) {
  boolean initial = MPU.enabled;
  MPU.enabled = enable.
  return initial;
}
```

Required when

The callback must be provided when memory protection is selected and there are OS Applications where TrustedApplicationWithProtection is true.

See Also

Os_Cbk_CheckMemoryAccess
CallTrustedFunction
CallAndProtectFunction
Os_Cbk_SetMemoryAccess

5.1.4 Os_Cbk_SetMemoryAccess

Callback routine used to prepare the memory protection system for a switch from trusted to access-restricted code (untrusted or trusted-with-protection).

Syntax

```
FUNC(void, {memclass})Os_Cbk_SetMemoryAccess(
    Os_UntrustedContextRefType ApplicationContext
)
```

Parameters

Name	Type	Mode	Description
`ApplicationContext`	`Os_UntrustedContextRefType`	in	A reference to a type that describes the untrusted context.

Description

This callback is provided so that you have full control over the memory protection hardware on your device, and so that you can decide the degree of protection that you want to apply on a particular project. For example, you may choose to limit write-access for access-restricted code but allow any read and execute access. Alternatively you may wish to limit read/write and execute access for access-restricted code.

In an AUTOSAR OS, code that runs in the context of a Trusted OS Application is assumed to have full access to any area of RAM, ROM or IO space that is available.

Such code runs in a privileged mode. On the other hand, code that runs in the context of an access-restricted OS Application may have restrictions placed on it that prevent it from being able access certain areas. Such code typically runs in 'user' (untrusted) mode, but there is also an AUTOSAR option to configure trusted OS Applications to run with memory protection enabled. Trusted-with-protection code behaves in most ways like trusted code. The only difference is that it runs with restricted access to memory.

Whenever RTA-OS is about to switch from trusted to access-restricted code, it makes a call to Os_Cbk_SetMemoryAccess. It passes in a reference to an Os_UntrustedContextType data structure that you can use to determine what permissions to set for access-restricted code. The Os_UntrustedContextType structure contains information about the OS Application, Task/ISR and stack region that applies to the code that is about to be executed. Depending on the context of the switch, some of these may contain NULL values. Os_Cbk_SetMemoryAccess is only called from trusted code.

Os_Cbk_SetMemoryAccess gets called in the following cases:

1) Before calling a TASK that belongs to an access-restricted OS-Application.

2) Before calling a Category 2 ISR that belongs to an access-restricted OS-Application.

3) Before calling an access-restricted OS-Application Startup, Shutdown or Error hook.

4) Before calling a 'TrustedFunction' that belongs to an access-restricted OS-Application. (This extends the AUTOSAR concept, and allows a core trusted task to call out to access-restricted code supplied by third parties.)

When using memory protection features, you must initialize the memory protection hardware before calling StartOS(). You can choose what hardware to use, how many regions to protect and what restrictions to apply.

If you want to run all access-restricted code with the same memory protection settings, then you can set the 'Single Memory Protection Zone' OS option. In this case Os_Cbk_SetMemoryAccess will not be called. You must set up the MPU before running any access-restricted code.

If you want to run all access-restricted code with the same basic memory protection settings but apply protection to the stack, then you can set the 'Stack Only Memory Protection' OS option. In this case Os_Cbk_SetMemoryAccess will only be passed the stack-related fields (Address and Size) plus Application. You must ensure that the memory protection settings limit the stack to the specified range.

* Note * Where the hardware does not allow protection regions to be set at any address/size combination, you may choose to adjust the stack to a position that can be protected efficiently. For example, the protection region may have to be aligned on a 64-byte address boundary. In these cases, RTA-OS provides the 'AlignUntrustedStacks' configuration option. When this is set, a further field 'ApplicationContext->AlignedAddress' becomes available. Its initial value will be the same as ApplicationContext->Address. However you can change its value to signal to the OS that the access-restricted code should start at a different location.

For the earlier example, if ApplicationContext->AlignedAddress initially has value 0x1020, you might change it to 0x1000 before returning so that the OS will start running the code at an address that is a multiple of 64. (This example assumes that the stack grows towards lower addresses.) You will have set the stack protection region to start from 0x1000.

* Note *

'FunctionID' and 'FunctionParams' are only present when there are access-restricted functions. The value of 'FunctionID' will be INVALID_FUNCTION except when the callback is for an access-restricted function. In this case, 'FunctionID' contains the function identifier and 'FunctionParams' is a copy of the pointer to the parameters of the function.

* Note *

'CoreID' is only present where there are multiple AUTOSAR cores, and it holds the number of the current core.

Note: memclass is OS_APPL_CODE for AUTOSAR 3.x, OS_OS_CBK_SETMEMORYACCESS_CODE for AUTOSAR 4.1 and OS_CALLOUT_CODE otherwise.

Example

```
FUNC(void, {memclass})
    Os_Cbk_SetMemoryAccess(Os_UntrustedContextRefType
    ApplicationContext) {
  /*
   * When called for an access-restricted TASK:
   *  ApplicationContext->Application contains the ID of the OS
       Application that the TASK belongs to.
   *  ApplicationContext->TaskID is the ID of the TASK
   *  ApplicationContext->ISRID is INVALID_ISR
   *  ApplicationContext->Address is the starting address for the
       TASK's stack.
   *  ApplicationContext->Size is the stack budget configured for
       the TASK. (Zero if no budget.)
   *  ApplicationContext->Trusted is true if the OS Application is
       trusted (TrustedApplicationWithProtection)
   *
   * When called for an access-restricted ISR:
   *  ApplicationContext->Application contains the ID of the OS
       Application that the ISR belongs to.
   *  ApplicationContext->TaskID is INVALID_TASK
   *  ApplicationContext->ISRID is the ID of the ISR
   *  ApplicationContext->Address is the starting address for the
       ISR's stack.
   *  ApplicationContext->Size is the stack budget configured for
       the ISR. (Zero if no budget.)
   *  ApplicationContext->Trusted is true if the OS Application is
       trusted (TrustedApplicationWithProtection)
   *
   * When called for:
   *    - an access-restricted Function
   *    - an access-restricted OS Application error hook
```

```
 *     - an access-restricted OS Application startup hook
 *     - an access-restricted OS Application shutdown hook
 *  ApplicationContext->Application contains the ID of the OS
     Application that the function/hook belongs to.
 *  ApplicationContext->TaskID is INVALID_TASK
 *  ApplicationContext->ISRID is INVALID_ISR
 *  ApplicationContext->Address is the value of the stack pointer
     just before the access-restricted code gets called.
 *  ApplicationContext->Size is zero
 *  ApplicationContext->Trusted is true if the OS Application is
     trusted (TrustedApplicationWithProtection)
 *
 * Where there are access-restricted Functions, there are two
     more fields:
 *  ApplicationContext->FunctionID contains the ID of the
     function (INVALID_FUNCTION unless being called for an
     access-restricted Function)
 *  ApplicationContext->FunctionParams contains FunctionParams
     for the access-restricted Function call (undefined for
     INVALID_FUNCTION)
 *
 * Be aware that on some target devices (Power PC, for example)
     the EABI might specify that a
 * back link will be written before the stack pointer on entry.
 * You will have to account for this in your calculations.
 *
 * For a multicore system, ApplicationContext->CoreID contains
     the ID of the calling core.
 * This is omitted if the OS is only running on one core.
 */

/*  Force AlignedAddress to the next 64-byte value below Address
    */
(uint32)ApplicationContext->AlignedAddress &=
    ((uint32)ApplicationContext->Address % 64U);
SET_STACK_RANGE(ApplicationContext->AlignedAddress,
    STACK_ALLOWANCE);

if (ApplicationContext->Application == App2) {
  /* Set memory protection regions that apply for the overall
      application 'App2' */
  SET_UNTRUSTED_WRITE_RANGE(App2_BASE, App2_SIZE); /* Example */
  if (ApplicationContext->TaskID == App2TaskB) {
    /* Extend or restrict ranges as desired for Task 'App2TaskB'
        */
  }
  if (ApplicationContext->ISRID == App2ISR1) {
    /* Extend or restrict ranges as desired for ISR 'App2ISR1' */
  }
  if (ApplicationContext->FunctionID == UTF1) {
    /* Extend or restrict ranges as desired for access-restricted
        Function 'tf1' */
  }
}
if (ApplicationContext->Application == App3) {
  /* Set memory protection regions that apply for the overall
      application 'App3' */
```

```
    SET_UNTRUSTED_WRITE_RANGE(App3_BASE, App3_SIZE); /* Example */
    if (ApplicationContext->TaskID == App3TaskB) {
      /* Extend or restrict ranges as desired for Task 'App3TaskB'
          */
    }
    if (ApplicationContext->FunctionID == UTF2) {
      /* Extend or restrict ranges as desired for access-restricted
          Function 'tf2' */
    }
    if (ApplicationContext->FunctionID == UTF3) {
      /* Extend or restrict ranges as desired for access-restricted
          Function 'tf3' */
    }
  }
  ...
}
OS_MAIN() {
  ...
  InitializeMemoryProtectionHardware();
  ...
  StartOS(OSDEFAULTAPPMODE);

}
```

Required when

The callback must be provided when memory protection is selected and there are access-restricted OS Applications.

See Also

Os_Cbk_CheckMemoryAccess
CallTrustedFunction
CallAndProtectFunction

5.2 Macros

5.2.1 CAT1_ISR

Macro that should be used to create a Category 1 ISR entry function. This macro exists to help make your code portable between targets.

Example

```
CAT1_ISR(MyISR) {...}
```

5.2.2 Os_DisableAllConfiguredInterrupts

The Os_DisableAllConfiguredInterrupts macro will disable all configured interrupts. You will need to #include the file "Os_DisableInterrupts.h" if you want to use this macro. It may not be used by untrusted code.

Example

```
Os_DisableAllConfiguredInterrupts()
Os_Enable_Millisecond()
```

5.2.3 Os_Disable_x

The Os_Disable_x macro will disable the named interrupt. It is normally paired with a call to Os_Enable_x. The macro can be called using either the vector address or the RTA-OS configured vector name. In the example, this is Os_Disable_12() and Os_Disable_Millisecond() respectively. You will need to #include the file "Os_DisableInterrupts.h" if you want to use these macros. They may not be used by untrusted code.

Example

```
Os_Disable_12()
Os_Disable_Millisecond()
```

5.2.4 Os_EnableAllConfiguredInterrupts

The Os_EnableAllConfiguredInterrupts macro will enable all configured interrupts. You will need to #include the file "Os_DisableInterrupts.h" if you want to use this macro. It may not be used by untrusted code.

Example

```
Os_DisableAllConfiguredInterrupts()
...
Os_EnableAllConfiguredInterrupts()
```

5.2.5 Os_Enable_x

The Os_Enable_x macro will re-enable the named interrupt. It is normally paired with a call to Os_Disable_x. The macro can be called using either the vector address or the RTA-OS configured vector name. In the example, this is Os_Enable_12() and Os_Enable_Millisecond() respectively. You will need to #include the file "Os_DisableInterrupts.h" if you want to use these macros. They may not be used by untrusted code.

Example

```
Os_Enable_12()
Os_Enable_Millisecond()
```

5.3 Type Definitions

5.3.1 Os_StackSizeType

An unsigned value representing an amount of stack in bytes.

Example

```
Os_StackSizeType stack_size;
stack_size = Os_GetStackSize(start_position, end_position);
```

5.3.2 Os_StackValueType

An unsigned value representing the position of the stack pointer (ESP).

Example

```
Os_StackValueType start_position;
start_position = Os_GetStackValue();
```

6 Toolchain

This chapter contains important details about RTA-OS and the Visual Studio or MinGW C/C++ toolchain. A port of RTA-OS is specific to both the target hardware and a specific version of the compiler toolchain. You must make sure that you build your application with the supported toolchain.

In addition to the version of the toolchain, RTA-OS may use specific tool options (switches). The options are divided into three classes:

kernel options are those used by **RTAOSGen** to build the RTA-OS kernel.

mandatory options must be used to build application code so that it will work with the RTA-OS kernel.

forbidden options must not be used to build application code.

Any options that are not explicitly forbidden can be used by application code providing that they do not conflict with the kernel and mandatory options for RTA-OS.

> **NOTE**
>
> ETAS has developed and tested RTA-OS using the tool versions and options indicated in the following sections. Correct operation of RTA-OS is only covered by the warranty in the terms and conditions of your deployment license agreement when using identical versions and options. If you choose to use a different version of the toolchain or an alternative set of options then it is your responsibility to check that the system works correctly. If you require a statement that RTA-OS works correctly with your chosen tool version and options then please contact ETAS to discuss validation possibilities.

6.1 Compiler Versions

This port of RTA-OS has been developed to work with the following compiler(s):

6.1.1 MinGW

Most recent versions of MinGW will work. VECUs are built for 32-bit targets.

Tested on	gcc.exe (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders) 12.2.0
See also	https://sourceforge.net/projects/mingw-w64/files/mingw-w64/mingw-w64-release/

6.1.2 VS2022

Microsoft Visual Studio 2022. Ensure that cl.exe is on the path and that Common7\Tools\VsDevCmd.bat has been run to initialize other environment variables

Tested on	Microsoft (R) C/C++ Optimizing Compiler Version 19.30.30705 for x86

6.1.3 VS2019

Microsoft Visual Studio 2019. Ensure that cl.exe is on the path and that Common7\Tools\VsDevCmd.bat has been run to initialize other environment variables

Tested on	Microsoft (R) C/C++ Optimizing Compiler Version 19.20.27508.1 for x86

6.1.4 VS2017

Microsoft Visual Studio 2017. Ensure that cl.exe is on the path and that Common7\Tools\VsDevCmd.bat has been run to initialize other environment variables

Tested on	Microsoft (R) C/C++ Optimizing Compiler Version 19.16.27024 for x86

6.1.5 VS2015

Microsoft Visual Studio 2015. Ensure that cl.exe is on the path and that Common7\Tools\vsvars32.bat has been run to initialize other environment variables

Tested on	Microsoft (R) C/C++ Optimizing Compiler Version 19.00.23506 for x86

6.1.6 VS2013

Microsoft Visual Studio 2013. Ensure that cl.exe is on the path and that Common7\Tools\vsvars32.bat has been run to initialize other environment variables

Tested on	Release tests are not run with this version.

6.1.7 VS2012

Microsoft Visual Studio 2012. Ensure that cl.exe is on the path and that Common7\Tools\vsvars32.bat has been run to initialize other environment variables

Tested on	Microsoft (R) C/C++ Optimizing Compiler Version 17.00.61030 for x86

If you require support for a compiler version not listed above, please contact ETAS.

6.2 Options used to generate this guide

6.2.1 Compiler (MinGW)

Name	gcc.exe
Version	Tested on 5.1.3 and 12.2.0

6.2.1.1 Options

Kernel Options

The following options were used to build the RTA-OS kernel for the configuration that was used to generate the performance figures in this document. If you select different target options, then the values used to build the kernel might change. You can run a Configuration Summary report to check the values used for your configuration.

`-m32` 32 bit i386 code

`-O2` Optimization level

6.2.2 Librarian (MinGW)

Name	ar.exe
Version	Tested on 5.1.3 and 12.2.0

6.2.3 Linker (MinGW)

Name	g++.exe
Version	Tested on 5.1.3 and 12.2.0

6.2.4 Compiler (VS2010)

Name	cl.exe
Version	16.00

6.2.4.1 Options

Kernel Options

The following options were used to build the RTA-OS kernel for the configuration that was used to generate the performance figures in this document. If you select different target options, then the values used to build the kernel might change. You can run a Configuration Summary report to check the values used for your configuration.

`/Z7` Include debug information

`/Od` No optimizations

`/EHsc` Exception handling

`/D_WIN32_WINNT=0x0400` Windows NT upwards

`/D_CRT_SECURE_NO_DEPRECATE` Eliminate deprecation warnings

`/MP` Build multiple source files concurrently.

6.2.5 Librarian (VS2010)

Name	lib.exe
Version	16.00

6.2.6 Linker (VS2010)

Name	cl.exe
Version	16.00

6.2.6.1 Options

6.2.7 Compiler (VS2012)

Name	cl.exe
Version	17.00

6.2.7.1 Options

Kernel Options

The following options were used to build the RTA-OS kernel for the configuration that was used to generate the performance figures in this document. If you select different target options, then the values used to build the kernel might change. You can run a Configuration Summary report to check the values used for your configuration.

`/Z7` Include debug information

`/Od` No optimizations

`/EHsc` Exception handling

`/D_WIN32_WINNT=0x0400` Windows NT upwards

`/D_CRT_SECURE_NO_DEPRECATE` Eliminate deprecation warnings

`/MP` Build multiple source files concurrently.

6.2.8 Librarian (VS2012)

Name	lib.exe
Version	17.00

6.2.9 Linker (VS2012)

Name	cl.exe
Version	17.00

6.2.9.1 Options

6.2.10 Compiler (VS2015)

Name	cl.exe
Version	19.00

6.2.10.1 Options

Kernel Options

The following options were used to build the RTA-OS kernel for the configuration that was used to generate the performance figures in this document. If you select different target options, then the values used to build the kernel might change. You can run a Configuration Summary report to check the values used for your configuration.

`/Z7` Include debug information

`/Od` No optimizations

`/EHsc` Exception handling

`/D_WIN32_WINNT=0x0400` Windows NT upwards

`/D_CRT_SECURE_NO_DEPRECATE` Eliminate deprecation warnings

`/MP` Build multiple source files concurrently.

6.2.11 Librarian (VS2015)

Name	lib.exe
Version	19.00

6.2.12 Linker (VS2015)

Name	cl.exe
Version	19.00

6.2.12.1 Options

6.2.13 Compiler (VS2017)

Name	cl.exe
Version	19.10

6.2.13.1 Options

Kernel Options

The following options were used to build the RTA-OS kernel for the configuration that was used to generate the performance figures in this document. If you select different target options, then the values used to build the kernel might change. You can run a Configuration Summary report to check the values used for your configuration.

`/Z7` Include debug information

`/Od` No optimizations

`/EHsc` Exception handling

`/D_WIN32_WINNT=0x0400` Windows NT upwards

`/D_CRT_SECURE_NO_DEPRECATE` Eliminate deprecation warnings

`/MP` Build multiple source files concurrently.

6.2.14 Librarian (VS2017)

Name	lib.exe
Version	19.10

6.2.15 Linker (VS2017)

Name	cl.exe
Version	19.10

6.2.15.1 Options

6.2.16 Compiler (VS2019)

Name	cl.exe
Version	19.20

6.2.16.1 Options

Kernel Options

The following options were used to build the RTA-OS kernel for the configuration that was used to generate the performance figures in this document. If you select different target options, then the values used to build the kernel might change. You can run a Configuration Summary report to check the values used for your configuration.

`/Z7` Include debug information

`/Od` No optimizations

`/EHsc` Exception handling

`/D_WIN32_WINNT=0x0400` Windows NT upwards

`/D_CRT_SECURE_NO_DEPRECATE` Eliminate deprecation warnings

`/MP` Build multiple source files concurrently.

6.2.17 Librarian (VS2019)

Name	lib.exe
Version	19.20

6.2.18 Linker (VS2019)

Name	cl.exe
Version	19.20

6.2.18.1 Options

6.2.19 Compiler (VS2022)

Name	cl.exe
Version	19.30

6.2.19.1 Options

Kernel Options

The following options were used to build the RTA-OS kernel for the configuration that was used to generate the performance figures in this document. If you select different target options, then the values used to build the kernel might change. You can run a Configuration Summary report to check the values used for your configuration.

`/Z7` Include debug information

`/Od` No optimizations

`/EHsc` Exception handling

`/D_WIN32_WINNT=0x0400` Windows NT upwards

`/D_CRT_SECURE_NO_DEPRECATE` Eliminate deprecation warnings

Mandatory Options for Application Code

The following options were mandatory for application code used with the configuration that was used to generate the performance figures in this document. If you select different target options, then the values required by application code might change. You can run a Configuration Summary report to check the values used for your configuration.

`/std:c11` Use C99 standards

6.2.20 Librarian (VS2022)

Name	lib.exe
Version	19.30

6.2.21 Linker (VS2022)

Name	cl.exe
Version	19.30

6.2.21.1 Options

7 Hardware

7.1 Supported Devices

This port of RTA-OS has been developed to work with the following target:

Name:	Microsoft Windows
Device:	Virtual ECU

The following variants of the Virtual ECU are supported:

- MinGW (MinGW GCC compiler) [Configurable number of cores]
- VS2010 (Microsoft Visual Studio 2010) [Configurable number of cores]
- VS2012 (Microsoft Visual Studio 2012) [Configurable number of cores]
- VS2015 (Microsoft Visual Studio 2015) [Configurable number of cores]
- VS2017 (Microsoft Visual Studio 2017) [Configurable number of cores]
- VS2019 (Microsoft Visual Studio 2019) [Configurable number of cores]
- VS2022 (Microsoft Visual Studio 2022) [Configurable number of cores]

If you require support for a variant of Virtual ECU not listed above, please contact ETAS.

7.2 Register Usage

7.2.1 Initialization

RTA-OS requires the following registers to be initialized to the indicated values before `StartOS()` is called.

Register	Setting
none	Initialization is done by the VRTA startup code

7.2.2 Modification

The following registers must not be modified by user code after the call to `StartOS()`:

Register	Notes
none	Not applicable

7.3 Required OS resources

RTA-OS needs the following resources for correct operation.

Resource	Description
Cross-core interrupt	In multi-core configurations, the OS will allocate one free interrupt to use for cross-core communication

7.4 Interrupts

This section explains the implementation of RTA-OS's interrupt model on the Virtual ECU.

7.4.1 Interrupt Priority Levels

Interrupts execute at an interrupt priority level (IPL). RTA-OS standardizes IPLs across all targets. IPL 0 indicates task level. IPL 1 and higher indicate an interrupt priority. It is important that you don't confuse IPLs with task priorities. An IPL of 1 is higher than the highest task priority used in your application.

The IPL is a target-independent description of the interrupt priority on your target hardware. The following table shows how IPLs are mapped onto the hardware interrupt priorities of the Virtual ECU:

IPL	ICU	Description
0	0	User (task) level
1-32	1-32	Category 1 and 2 level

Even though a particular mapping is permitted, all Category 1 ISRs must have equal or higher IPL than all of your Category 2 ISRs.

7.4.2 Allocation of ISRs to Interrupt Vectors

The following restrictions apply for the allocation of Category 1 and Category 2 interrupt service routines (ISRs) to interrupt vectors on the Virtual ECU. A ✓ indicates that the mapping is permitted and a ✗ indicates that it is not permitted:

Address	Category 1	Category 2
1-32 (1-31 when using multiple cores)	✓	✓

7.4.3 Vector Table

RTAOSGen normally generates an interrupt vector table for you automatically. You can configure "Suppress Vector Table Generation" as `true` to stop RTA-OS from generating the interrupt vector table.

Depending upon your target, you may be responsible for locating the generated vector table at the correct base address. The following table shows the section (or sections) that need to be located and the associated valid base address:

Section	Valid Addresses
none	Initialization is done by the VRTA startup code.

7.4.4 Writing Category 1 Interrupt Handlers

Raw Category 1 interrupt service routines (ISRs) must correctly handle the interrupt context themselves. RTA-OS provides an optional helper macro `CAT1_ISR` that can be used to make code more portable. Depending on the target, this may cause the selection of an appropriate interrupt control directive to indicate to the compiler that a function requires additional code to save and restore the interrupt context.

A Category 1 ISR therefore has the same structure as a Category 2 ISR, as shown below.

```
CAT1_ISR(Category1Handler) {
  /* Handler routine */
}
```

7.4.5 Writing Category 2 Interrupt Handlers

Category 2 ISRs are provided with a C function context by RTA-OS, since the RTA-OS kernel handles the interrupt context itself. The handlers are written using the `ISR()` macro as shown below:

```
#include <Os.h>
ISR(MyISR) {
  /* Handler routine */
}
```

You must not insert a return from interrupt instruction in such a function. The return is handled automatically by RTA-OS.

7.4.6 Default Interrupt

The 'default interrupt' is intended to be used to catch all unexpected interrupts. All unused interrupts have their interrupt vectors directed to the named routine that you specify. The routine you provide is not handled by RTA-OS and must correctly handle the interrupt context itself. The handler must use the `CAT1_ISR` macro in the same way as a Category 1 ISR (see Section 7.4.4 for further details).

7.5 Memory Model

The following memory models are supported:

Model	Description
32 bit	VRTA is a 32-bit Windows application

7.6 Processor Modes

RTA-OS can run in the following processor modes:

Mode	Notes
Standard	VRTA runs as a standard Windows executable

7.7 Stack Handling

RTA-OS uses a single stack for all tasks and ISRs.

No special stack configuration is required for the VRTA port.

8 Performance

This chapter provides detailed information on the functionality, performance and memory demands of the RTA-OS kernel. RTA-OS is highly scalable. As a result, different figures will be obtained when your application uses different sets of features. The figures presented in this chapter are representative for the VRTA port based on the following configuration:

- There are 32 tasks in the system
- Standard build is used
- Stack monitoring is disabled
- Time monitoring is disabled
- There are no calls to any hooks
- Tasks have unique priorities
- Tasks are not queued (i.e. tasks are BCC1 or ECC1)
- All tasks terminate/wait in their entry function
- Tasks and ISRs do not save any auxiliary registers (for example, floating point registers)
- Resources are shared by tasks only
- The generation of the resource `RES_SCHEDULER` is disabled

8.1 Measurement Environment

The following hardware environment was used to take the measurements in this chapter:

Device	VS2022 on Pentium class PC
CPU Clock Speed	2000.0MHz
Stopwatch Speed	1000.0MHz

8.2 RAM and ROM Usage for OS Objects

Each OS object requires some ROM and/or RAM. The OS objects are generated by **RTAOSGen** and placed in the RTA-OS library. In the main:

- `Os_Cfg_Counters` includes data for counters, alarms and schedule tables.
- `Os_Cfg` contains the data for most other OS objects.

8.2.1 Single Core

The following table gives the ROM and/or RAM requirements (in bytes) for each OS object in a simple single-core configuration. Note that object sizes will vary depending on the project configuration and compiler packing issues.

Object	ROM	RAM
Alarm	2	16
Cat 2 ISR	16	4
Counter	20	4
CounterCallback	7.5	0
ExpiryPoint	29	0
OS Overheads (max)	0	1060
OS-Application	0	0
PeripheralArea	0	0
Resource	8	8
ScheduleTable	16	16
Task	28	4

8.2.2 Multi Core

The following table gives the ROM and/or RAM requirements (in bytes) for each OS object in a simple multi-core configuration. Note that object sizes will vary depending on the project configuration and compiler packing issues.

Object	ROM	RAM
Alarm	4	16
Cat 2 ISR	20	4
Core Overheads (each OS core)	0	56
Core Overheads (each processor core)	20	1233
Counter	28	4
CounterCallback	7.5	0
ExpiryPoint	50	0
OS Overheads (max)	0	6
OS-Application	2.66	0
PeripheralArea	0	0
Resource	12	8
ScheduleTable	16	16
Task	36	4

8.3 Stack Usage

The amount of stack used by each Task/ISR in RTA-OS is equal to the stack used in the Task/ISR body plus the context saved by RTA-OS. The size of the run-time context saved by RTA-OS depends on the Task/ISR type and the exact system configuration. The only reliable way to get the correct value for Task/ISR stack usage is to call the `Os_GetStackUsage()` API function.

Note that because RTA-OS uses a single-stack architecture, the run-time contexts of all tasks reside on the same stack and are recovered when the task terminates.

As a result, run-time contexts of mutually exclusive tasks (for example, those that share an internal resource) are effectively overlaid. This means that the worst case stack usage can be significantly less than the sum of the worst cases of each object on the system. The RTA-OS tools automatically calculate the total worst case stack usage for you and present this as part of the configuration report.

8.4 Library Module Sizes

8.4.1 Single Core

The RTA-OS kernel is demand linked. This means that each API call is placed into a separately linkable module. The following table lists the section sizes for each API module (in bytes) for the simple single-core configuration in standard status.

Library Module	.bss	.data	.rdata	.text	.voltbl	.xdata
ActivateTask				255	36	
AdvanceCounter				50	16	
CallTrustedFunction				91	16	
CancelAlarm				152	16	
ChainTask				171	36	
CheckISRMemoryAccess				109	16	
CheckObjectAccess				336	16	
CheckObjectOwnership				292	16	
CheckTaskMemoryAccess				109	16	
ClearEvent				80	16	
ControlIdle	4			115	16	
DisableAllInterrupts	8			102	32	
DispatchTask				303	104	
ElapsedTime				560	112	
EnableAllInterrupts				90	32	
GetActiveApplicationMode				66	20	
GetAlarm				237	16	
GetAlarmBase				100	16	
GetApplicationID				110	28	
GetCounterValue				92	16	
GetCurrentApplicationID				110	28	
GetElapsedCounterValue				150	16	
GetEvent				80	16	
GetExecutionTime				90	16	
GetISRID				65	20	
GetIsrMaxExecutionTime				90	16	
GetIsrMaxStackUsage				90	16	
GetResource				166	40	

Library Module	.bss	.data	.rdata	.text	.voltbl	.xdata
GetScheduleTableStatus				93	16	
GetStackSize				65	16	
GetStackUsage				90	16	
GetStackValue				70	16	
GetTaskID				62	20	
GetTaskMaxExecutionTime				90	16	
GetTaskMaxStackUsage				90	16	
GetTaskState				108	24	
GetVersionInfo				101	16	
Idle				7		
InShutdown				5		
IncrementCounter				60	16	
InterruptSource			4	368	48	
ModifyPeripheral				320	48	
NextScheduleTable				233	16	
Os_Cbk_ChangedTrust				5		
Os_Cbk_ChangingTrust				5		
Os_Cfg	885		1040	405	32	
Os_Cfg_Counters			728	9705	384	
Os_Cfg_KL				108	32	
Os_GetCurrentIMask				33	16	
Os_GetCurrentTPL				96	24	
Os_Metrics	992			92		
Os_SetJmp				62		
Os_Stack	4			112	16	
Os_Trust				258		
Os_fsb				4		
Os_vectors			388			
Os_wrapper				214	52	
ProtectionSupport				55		
ReadPeripheral				282	48	
ReleaseResource				188	28	
ResetIsrMaxExecutionTime				80	16	
ResetIsrMaxStackUsage				80	16	
ResetTaskMaxExecutionTime				80	16	
ResetTaskMaxStackUsage				80	16	
ResumeAllInterrupts				90	32	
ResumeOSInterrupts				98	40	
Schedule				194	44	
SetAbsAlarm				188	16	

Library Module	.bss	.data	.rdata	.text	.voltbl	.xdata
SetEvent				80	16	
SetRelAlarm				246	16	
SetScheduleTableAsync				121	16	
ShutdownOS				104	28	
StackOverrunHook				15		
StartOS				206	40	
StartScheduleTableAbs				245	16	
StartScheduleTableRel				204	16	
StartScheduleTableSynchron				121	16	
StopScheduleTable				142	16	
SuspendAllInterrupts	8			102	32	
SuspendOSInterrupts	8			187	52	
SyncScheduleTable				123	16	
SyncScheduleTableRel				123	16	
TerminateTask				70	24	
TestErrorHook				14	12	
ValidateCounter				122		
ValidateISR				71		
ValidateResource				122		
ValidateScheduleTable				122		
ValidateTask				81		
WaitEvent				80	16	
WritePeripheral				280	48	
target				405		
virtualDevice		183	557	8486	212	692
vrtaAppMain				222		
vrtaAppSupport	68		66	509	80	
vrtaCore	176	29	892	4950	84	296
vrtaIdleHandling	4	33	77	218	16	
vrtaInProcessAPI	13	26		764	68	
vrtaLoggerDevice	8	46	933	3774	16	
vrtaOSMonitorDevice	64	134	3997	20963	177	104
vrtaPrintf	20580		17	120		
vrtaSampleDevices		493	7041	8734	248	124
vrtaSystemClock	52	167	296	4509	175	212
vrtaSystemClockDevice	344	130	2318	5382	9	52

8.4.2 Multi Core

The RTA-OS kernel is demand linked. This means that each API call is placed into a separately linkable module. The following table lists the section sizes for each API module (in bytes) for the simple multi-core configuration in standard status.

Library Module	.bss	.data	.rdata	.text	.voltbl	.xdata
ActivateTask				657	56	
AdvanceCounter				88	16	
CallTrustedFunction				126	16	
CancelAlarm				275	24	
ChainTask				305	40	
CheckISRMemoryAccess				150	16	
CheckObjectAccess			3	572	16	
CheckObjectOwnership				364	16	
CheckTaskMemoryAccess				150	16	
ClearEvent				115	16	
ControlIdle	8			161	16	
CrossCore				165	16	
DisableAllInterrupts				139	32	
DispatchTask				892	216	
ElapsedTime				851	112	
EnableAllInterrupts				139	32	
GetActiveApplicationMode				101	20	
GetAlarm				271	16	
GetAlarmBase				138	16	
GetApplicationID				196	40	
GetCounterValue				127	16	
GetCurrentApplicationID				196	40	
GetElapsedCounterValue				183	16	
GetEvent				115	16	
GetExecutionTime				123	16	
GetISRID				108	20	
GetIsrMaxExecutionTime				123	16	
GetIsrMaxStackUsage				123	16	
GetNumberOfActivatedCores				154	24	
GetResource				208	40	
GetScheduleTableStatus				208	20	
GetSpinlock				90	16	
GetStackSize				100	16	
GetStackUsage				123	16	
GetStackValue				113	16	

Library Module	.bss	.data	.rdata	.text	.voltbl	.xdata
GetTaskID				109	20	
GetTaskMaxExecutionTime				123	16	
GetTaskMaxStackUsage				123	16	
GetTaskState				263	32	
GetVersionInfo				139	16	
Idle				7		
InShutdown				5		
IncrementCounter				98	16	
InterruptSource			4	481	48	
ModifyPeripheral				419	48	
NextScheduleTable				356	24	
Os_Cbk_ChangedTrust				5		
Os_Cbk_ChangingTrust				5		
Os_Cfg	3424		1488	597	44	
Os_Cfg_Counters			888	13936	536	
Os_Cfg_KL				393	48	
Os_GetCurrentIMask				60	16	
Os_GetCurrentTPL				234	32	
Os_Locking				8		
Os_Metrics				160		
Os_SetJmp				62		
Os_Stack	8			112	16	
Os_Trust				419		
Os_fsb				4		
Os_vectors			388	268	36	
Os_wrapper				394	64	
ProtectionSupport				82		
ReadPeripheral				381	48	
ReleaseResource				232	28	
ReleaseSpinlock				90	16	
ResetIsrMaxExecutionTime				115	16	
ResetIsrMaxStackUsage				115	16	
ResetTaskMaxExecutionTime				115	16	
ResetTaskMaxStackUsage				115	16	
ResumeAllInterrupts				139	32	
ResumeOSInterrupts				145	40	
Schedule				239	44	
SetAbsAlarm				311	24	
SetEvent				121	16	
SetRelAlarm				372	24	

Library Module	.bss	.data	.rdata	.text	.voltbl	.xdata
SetScheduleTableAsync				154	16	
ShutdownAllCores				174	16	
ShutdownOS				187	28	
StackOverrunHook				15		
StartCore				139	20	
StartOS				515	72	
StartScheduleTableAbs				368	24	
StartScheduleTableRel				327	24	
StartScheduleTableSynchron				154	16	
StopScheduleTable				265	24	
SuspendAllInterrupts				139	32	
SuspendOSInterrupts				224	52	
SyncScheduleTable				156	16	
SyncScheduleTableRel				156	16	
TerminateTask				116	24	
TestErrorHook				19	12	
TryToGetSpinlock				99	16	
ValidateCounter				122		
ValidateISR				71		
ValidateResource				122		
ValidateScheduleTable				122		
ValidateTask				155		
WaitEvent				115	16	
WritePeripheral				379	48	
target				437		
virtualDevice		183	557	8486	212	692
vrtaAppMain				247		
vrtaAppSupport	70		66	509	80	
vrtaCore	180	29	892	4950	84	296
vrtaIdleHandling	4	33	77	250	16	
vrtaInProcessAPI	13	26		789	68	
vrtaLoggerDevice	8	46	933	3774	16	
vrtaOSMonitorDevice	68	134	4027	21907	179	112
vrtaPrintf	20580		17	120		
vrtaSampleDevices		493	7041	8734	248	124
vrtaSystemClock	56	167	296	4554	179	212
vrtaSystemClockDevice	344	130	2318	5382	9	52

8.5 Execution Time

The following tables give the execution times in CPU cycles, i.e. in terms of ticks of the processor's program counter. These figures will normally be independent of the frequency at which you clock the CPU. To convert between CPU cycles and SI time units the following formula can be used:

Time in microseconds = Time in cycles / CPU Clock rate in MHz

For example, an operation that takes 50 CPU cycles would be:

- at 20MHz = 50/20 = 2.5μs
- at 80MHz = 50/80 = 0.625μs
- at 150MHz = 50/150 = 0.333μs

While every effort is made to measure execution times using a stopwatch running at the same rate as the CPU clock, this is not always possible on the target hardware. If the stopwatch runs slower than the CPU clock, then when RTA-OS reads the stopwatch, there is a possibility that the time read is less than the actual amount of time that has elapsed due to the difference in resolution between the CPU clock and the stopwatch (the **User Guide** provides further details on the issue of uncertainty in execution time measurement).

The figures presented in Section 8.5.1 have an uncertainty of 1 CPU cycle(s).

Values are given for single-core operation only. Timings for cross-core activations, though interesting, are variable because of the nature of multi-core operation. Minimum values cannot be given, because timings are dependent on the activity on the core that receives the activation.

8.5.1 Context Switching Time

Task switching time is the time between the last instruction of the previous task and the first instruction of the next task. The switching time differs depending on the switching contexts (e.g. an `ActivateTask()` versus a `ChainTask()`).

Interrupt latency is the time between an interrupt request being recognized by the target hardware and the execution of the first instruction of the user provided handler function:

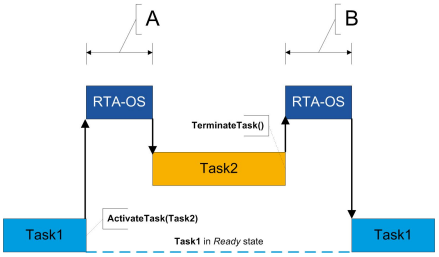
For Category 1 ISRs this is the time required for the hardware to recognize the interrupt.

For Category 2 ISRs this is the time required for the hardware to recognize the interrupt plus the time required by RTA-OS to set-up the context in which the ISR runs.

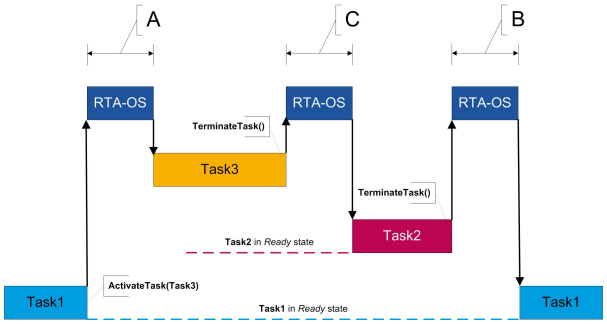
Figure 8.1 shows the measured context switch times for RTA-OS.

Switch	Key	CPU Cycles	Actual Time

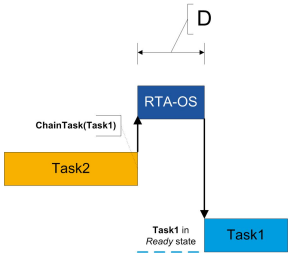
Task activation	A	- not available -	- not available -
Task termination with resume	B	- not available -	- not available -
Task termination with switch to new task	C	- not available -	- not available -
Chaining a task	D	- not available -	- not available -
Waiting for an event resulting in transition to the WAITING state	E	- not available -	- not available -
Setting an event results in task switch	F	- not available -	- not available -
Non-preemptive task offers a preemption point (co-operative scheduling)	G	- not available -	- not available -
Releasing a resource results in a task switch	H	- not available -	- not available -
Entering a Category 2 ISR	I	- not available -	- not available -
Exiting a Category 2 ISR and resuming the interrupted task	J	- not available -	- not available -
Exiting a Category 2 ISR and switching to a new task	K	- not available -	- not available -
Entering a Category 1 ISR	L	- not available -	- not available -



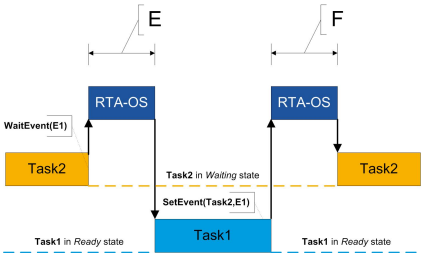
(a) Task activated. Termination resumes preempted task.



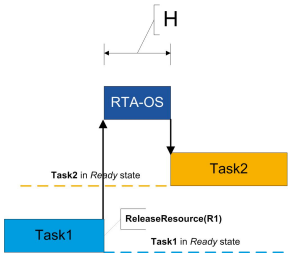
(b) Task activated. Termination switches into new task.



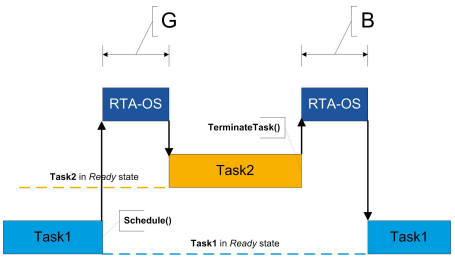
(c) Task chained.



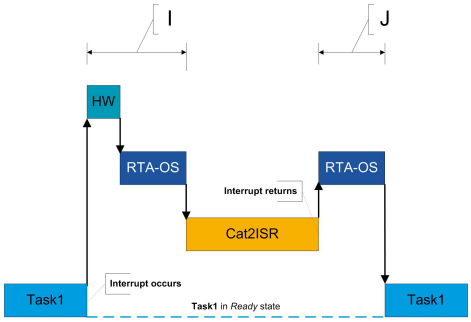
(d) Task waits. Task is resumed when event set.



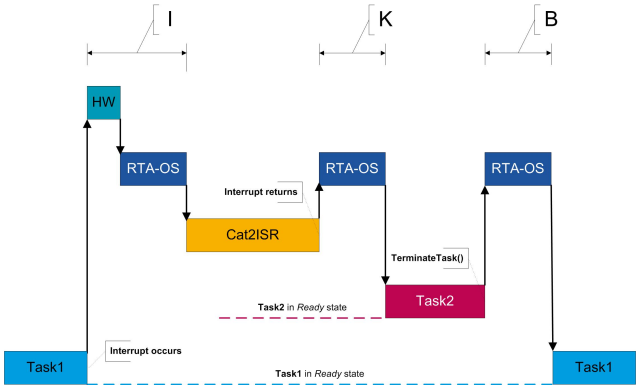
(e) Task switch when resource is released.



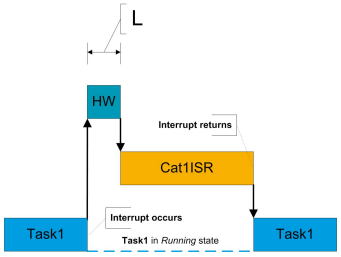
(f) Request for scheduling made by non-preemptive task.



(g) Category 2 interrupt entry. Interrupted task resumed on exit.



(h) Category 2 interrupt entry. Switch to new task on exit.



(i) Category 1 interrupt entry.

Figure 8.1: Context Switching

9 Finding Out More

Additional information about VRTA-specific parts of RTA-OS can be found in the following manuals:

VRTA Release Note. This document provides information about the VRTA port plug-in release, including a list of changes from previous releases and a list of known limitations.

Virtual ECU User Guide. This guide explains how to use the Virtual ECU environment included with the VRTA port plug-in.

Information about the port-independent parts of RTA-OS can be found in the following manuals, which can be found in the RTA-OS installation (typically in the Documents folder):

Getting Started Guide. This document explains how to install RTA-OS tools and describes the underlying principles of the operating system

Reference Guide. This guide provides a complete reference to the API, programming conventions and tool operation for RTA-OS.

User Guide. This guide shows you how to use RTA-OS to build real-time applications.

10 Contacting ETAS

10.1 Technical Support

Technical support is available to all users with a valid support contract. If you do not have a valid support contract, please contact your regional sales office (see below).

The best way to get technical support is by email. Any problems or questions about the use of the product should be sent to:

rta.hotline@etas.com

If you prefer to discuss your problem with the technical support team, you call the support hotline on:

+44 (0)1904 562624.

The hotline is available during normal office hours (0900-1730 GMT/BST).

In either case, it is helpful if you can provide technical support with the following information:

- Your support contract number
- The version of the ETAS tools you are using
- The version of the compiler toolchain you are using
- The command line (or reproduction of steps) that result in an error message
- The error messages or return codes you received (if any)
- Your `.xml`, `.arxml` and `.rtaos` files
- The file `Diagnostic.dmp` if it was generated

10.2 General Enquiries

10.2.1 ETAS Global Headquarters

ETAS GmbH		
Borsigstrasse 24	Phone:	+49 711 3423-0
70469 Stuttgart	Fax:	+49 711 3423-2106
Germany	WWW:	www.etas.com

10.2.2 ETAS Local Sales & Support Offices

Contact details for your local sales office and local technical support team (where available) can be found on the ETAS web site:

ETAS subsidiaries	www.etas.com/en/contact.php
ETAS technical support	www.etas.com/en/hotlines.php

Index

A

AUTOSAR OS includes
- Os.h, 25
- Os_Cfg.h, 25
- Os_MemMap.h, 25

C

CAT1_ISR, 33
Compiler (MinGW), 36, 37
Compiler (VS2010), 38
Compiler (VS2012), 37, 39
Compiler (VS2013), 37
Compiler (VS2015), 37, 40
Compiler (VS2017), 37, 40
Compiler (VS2019), 37, 41
Compiler (VS2022), 36, 42
Compiler Versions, 36
Configuration
- Port-Specific Parameters, 23

E

ETAS License Manager, 13
- Installation, 13

F

Files, 25

H

Hardware
- Requirements, 11

I

Installation, 11
- Default Directory, 12
- Verification, 20

Interrupts, 45
- Category 1, 46
- Category 2, 46
- Default, 46

IPL, 45

L

Librarian (MinGW), 38
Librarian (VS2010), 39
Librarian (VS2012), 39
Librarian (VS2015), 40
Librarian (VS2017), 41
Librarian (VS2019), 42
Librarian (VS2022), 43
Library
- Name of, 25

License, 12
- Borrowing, 17
- Concurrent, 14
- Grace Mode, 14
- Installation, 16
- Machine-named, 14
- Status, 17
- Troubleshooting, 18
- User-named, 14

Linker (MinGW), 38
Linker (VS2010), 39
Linker (VS2012), 39
Linker (VS2015), 40
Linker (VS2017), 41
Linker (VS2019), 42
Linker (VS2022), 43

M

Memory Model, 47

O

Options, 37
Os_Cbk_ChangedTrust, 26
Os_Cbk_ChangingTrust, 27
Os_Cbk_GetSetProtection, 28
Os_Cbk_SetMemoryAccess, 29
Os_Disable_x, 34
Os_DisableAllConfiguredInterrupts, 33
Os_Enable_x, 34
Os_EnableAllConfiguredInterrupts, 34
Os_StackSizeType, 34
Os_StackValueType, 35

P

Parameters of Implementation, 22
Performance, 48
- Context Switching Times, 56
- Library Module Sizes, 50
- RAM and ROM, 48
- Stack Usage, 49

Processor Modes, 47
- Standard, 47

R

Registers
- Initialization, 44
- Non-modifiable, 44
- none, 44

Resource
- Cross-core interrupt, 45

Run-time library, 22

S

Software
    Requirements, 11
Stack, 47

T
Target, 44
    Variants, 44
Toolchain, 36

V
Variants, 44
Vector Table
    Base Address, 46